T. Clase 01

TALLER PRÁCTICO: Desarmando Gigantes - Ingeniería Inversa en la Práctica

Integrantes:

- Cuasapaz Guilcamaigua Jonathan Franklin
- Guamán Maza Jonathan Steeven
- Sigcha Díaz Christian David

Carrera: Ingeniería de Sistemas / Software

Materia: Construcción y Evolución de Software

Capítulo 1: Ingeniería inversa, mantenimiento y evolución

Tópico: 1.1 ¿Qué es Ingeniería Inversa? Técnicas, herramientas y futuro.

Objetivo del Taller

Que el estudiante apropie los conceptos fundamentales de la ingeniería inversa, analice su evolución histórica y tecnológica, y aplique estos conocimientos para hipotetizar el funcionamiento de una plataforma de software moderna de alto impacto, utilizando Inteligencia Artificial como asistente de investigación y generación de contenido.

Aquí tienes el contenido ampliado y detallado para tu trabajo, Chris. Se ha profundizado en los conceptos teóricos, la justificación técnica de la arquitectura y se ha extendido el guion para que la presentación tenga un tono mucho más profesional y convincente.

Fase 1: Calentamiento

- **Ingeniería Inversa:** Es pasar del *qué hace* al *cómo lo hace* sin tener acceso a los planos originales o al código fuente. En la industria, esto es como recibir un pastel terminado y tratar de deducir la receta exacta probando sus ingredientes, analizando su textura y horneado.
- **Mantenimiento de Software:** Existen tipos preventivos, correctivos, adaptativos y perfectivos. La ingeniería inversa es vital aquí porque muchas veces el código fuente original se pierde, la documentación es nula, o los desarrolladores originales ya no están en la empresa. Permite mapear dependencias ocultas y entender el sistema actual para poder aplicar parches de seguridad o migrarlo a la nube sin romper funciones críticas.
- **Evolución del Software:** Ayuda a evitar la obsolescencia tecnológica. Permite extraer la lógica de negocio de un sistema antiguo para reescribirla en arquitecturas modernas (como microservicios). La ingeniería inversa permite identificar qué partes del código son esenciales, permitiendo que el sistema mute y escale sin perder su esencia operativa original.

Fase 2: Investigación Asistida por IA

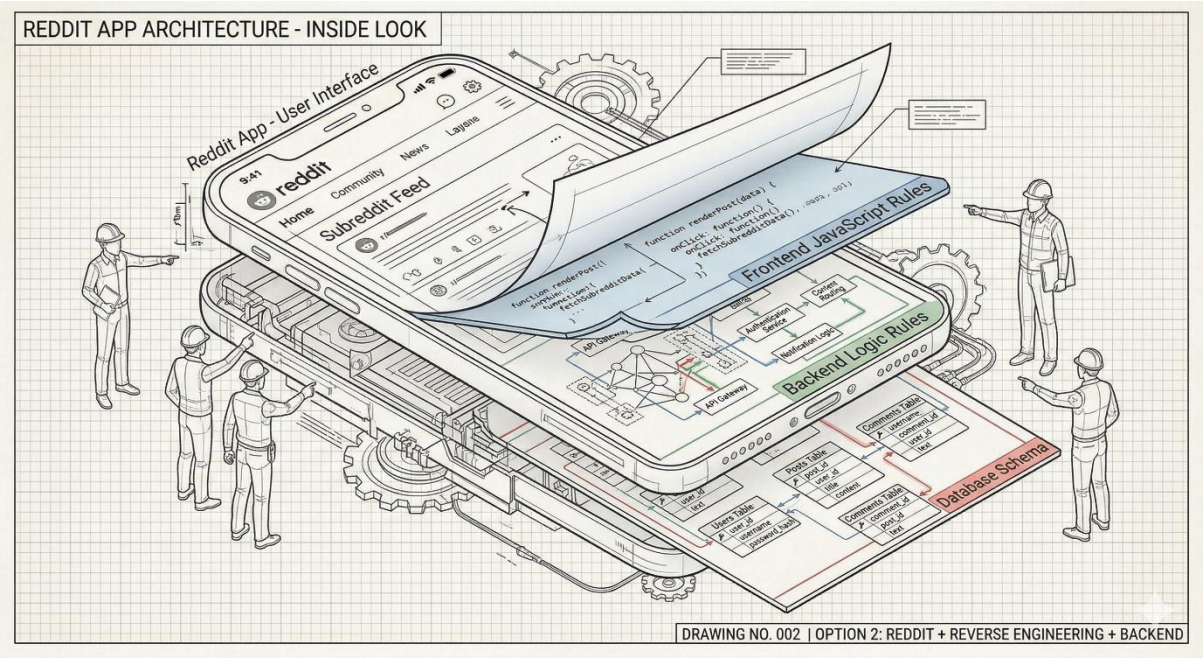
Evolución de la Ingeniería Inversa: Antes, Ahora y Mañana

- **El "Antes" (Años 90 - Sistemas Legacy y Monolíticos):**
  - *Entorno:* Binarios monolíticos instalados localmente, aplicaciones de escritorio y sistemas operativos sin conexión a internet constante.
  - *Herramientas:* Desensambladores, *debuggers* dinámicos (OllyDbg, SoftICE) y editores hexadecimales.
  - *Técnica principal:* Ingeniería inversa estática y dinámica pura. Los ingenieros leían lenguaje ensamblador crudo para entender el flujo de ejecución a nivel de registros de CPU y modificaban los binarios para saltarse rutinas de validación de licencias o entender formatos de archivos .
- **El "Ahora" (Cloud Computing, APIs y Microservicios):**
  - *Entorno:* Sistemas distribuidos (SaaS), ofuscación de código avanzada, aplicaciones móviles encriptadas y comunicación constante en red.
  - *Herramientas:* Proxies HTTP/HTTPS (Burp Suite, Charles Proxy), analizadores de protocolos (Wireshark), descompiladores de bytecode (JADX para Android) y herramientas de inspección web (DevTools).
  - *Técnica principal: API Sniffing* (interceptar y descifrar el tráfico de red) y pruebas de *Caja Negra*. Hoy en día el "cerebro" (backend) no se puede descargar. La arquitectura se infiere analizando cómo responde el servidor a diferentes *payloads* JSON, inyecciones de datos y variaciones en los encabezados HTTP.
- **Las "Novedades" (La era de la Inteligencia Artificial Generativa):**
  - *Entorno:* Automatización masiva y análisis semántico de código masivo.
  - *Herramientas:* LLMs especializados en programación (Claude, ChatGPT), Copilots de análisis estático y motores de IA integrados en herramientas como Ghidra.
  - *Técnica principal:* Descompilación semántica y traducción asistida. La IA toma código ensamblador incomprensible, binarios ofuscados o tráfico de red encriptado y genera pseudo-código estructurado en Python o C. Asigna nombres lógicos a las variables y funciones deducidos por el contexto, haciendo legible en minutos lo que antes tomaba semanas de frustrante análisis manual.

Fase 3: El Caso de Estudio - "Hacking Hipotético" a Reddit

- **Selección del Objetivo:** El algoritmo del feed "Hot" (Popular) y la sincronización en tiempo real del sistema de Karma (Upvotes/Downvotes) en la plataforma Reddit.
- **Identificación de Técnicas de Ingeniería Inversa:**
  - *API Sniffing (Análisis de Tráfico Activo):* Desplegar un proxy In-The-Middle para interceptar peticiones WebSockets. El objetivo es capturar la estructura exacta del JSON que se envía al dar un upvote (por ejemplo, descubrir variables ocultas y mapear el endpoint de destino (ej. `graphql.reddit.com/vote`).
  - *Caja Negra (Aislamiento de Variables):* Crear un entorno de pruebas controlado con múltiples cuentas. Manipular el tiempo exacto de publicación y la velocidad de votos iniciales (5 votos en 1 minuto vs. 5 votos en 1 hora). Al medir cómo esto afecta la posición en el feed de "Hot", se deduce la fórmula matemática subyacente: un algoritmo de decaimiento logarítmico basado en el tiempo de vida del post.
  - *Ingeniería Inversa del Cliente Web :* Descargar los *bundles* de JavaScript de la aplicación cliente (React). Aunque el código esté minificado, se pueden buscar patrones de cadenas de texto para mapear cómo la interfaz grafica reacciona a los eventos de Server-Sent Events (SSE) sin recargar la página.
- **Hipótesis de la Arquitectura (Diseño para el Diagrama Lógico):**
  - *Frontend:* Cliente Single Page Application (ReactJS) o App Móvil nativa manteniendo una conexión persistente vía WebSockets.
  - *API Gateway / Load Balancer:* Una capa externa (ej. AWS API Gateway o Nginx) que recibe millones de peticiones por segundo y actúa como escudo contra ataques DDoS, enrutando el tráfico limpio a la red interna.
  - *Microservicios Desacoplados:*
    - **Servicio de Votación:** Una API hiper-ligera (Node.js/Go) cuya única función es validar el token del usuario e inyectar el voto rápidamente a la cola.
    - **Servicio de Ranking:** Un motor de procesamiento matemático (Python/C++) que recalcula constantemente el puntaje de cada hilo.
  - *Capa de Persistencia y Mensajería (El núcleo del sistema):*
    - **Apache Kafka (Cola de Mensajes):** Absorbe el impacto. No se puede escribir en la base de datos con cada clic. Kafka recibe todos los votos y los encola ordenadamente.

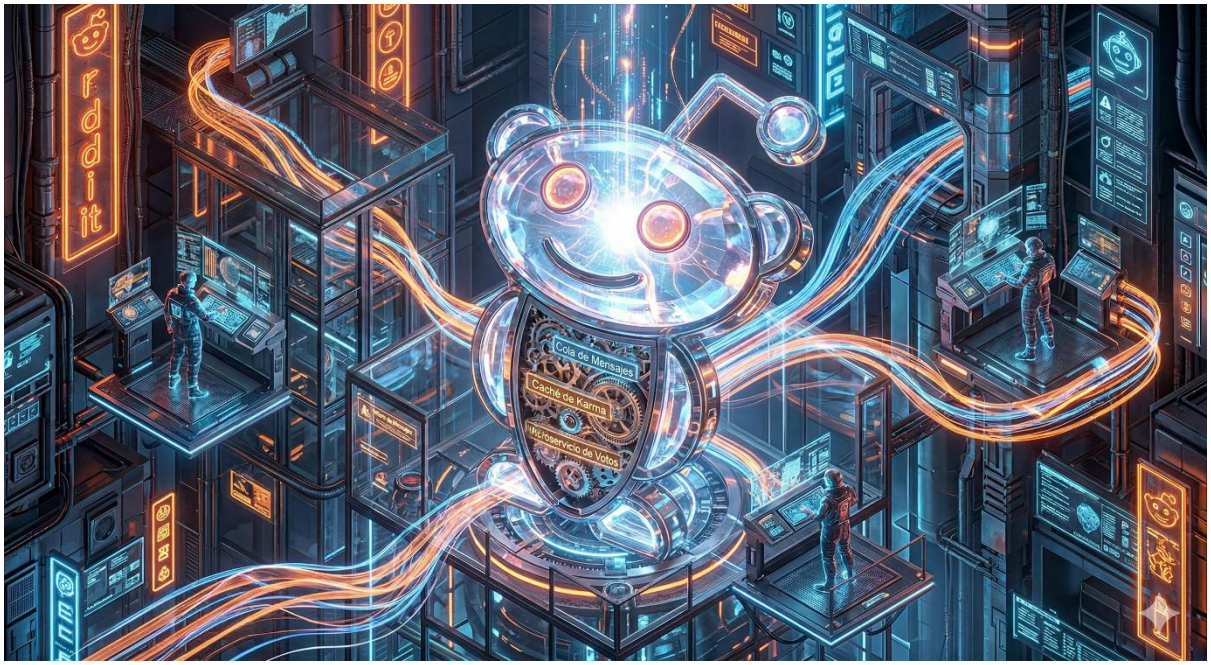
- **Redis (Caché en Memoria):** Lee los datos de Kafka para actualizar los contadores visuales de Karma en los perfiles y posts en milisegundos.
- **PostgreSQL (Base de Datos Primaria):** Almacena el registro histórico e inmutable de forma asíncrona mediante procesos por lotes (workers), asegurando la consistencia de los datos a largo plazo.

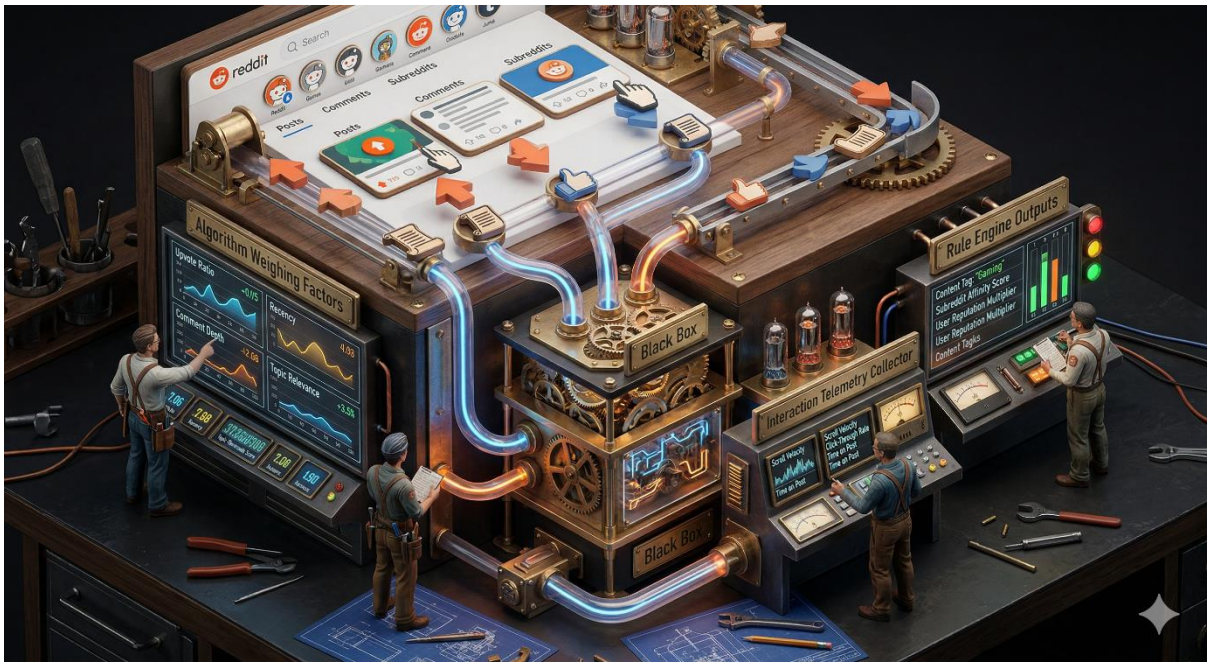


Fase 4: Generación de Imagen para la Exposición

Entregable (Opción Principal - Estilo Cyberpunk):

"Un render 3D de estilo isométrico cyberpunk que muestra el icónico logo del alien de Reddit gigante funcionando como el núcleo de un servidor de cristal. Unos ingenieros holográficos están interceptando flujos de datos naranjas y azules (representando upvotes y downvotes) utilizando herramientas de análisis de red. Dentro del núcleo de cristal abierto se revelan engranajes brillantes etiquetados textualmente como 'Logarithmic Decay', 'Message Queues' y 'Karma Cache'. Iluminación neón naranja y azul, hiperdetallado, renderizado en motor unreal engine 5, estética de hacking arquitectónico."



Entregable (Opción Alternativa - Estilo Blueprint Técnico):

"Una ilustración digital de estilo 'blueprint' arquitectónico azul oscuro de una granja de servidores de Reddit. Se ve el flujo esquemático de un 'Upvote' transformándose en una onda de energía logarítmica. Las tuberías de datos están rotuladas claramente como 'WebSockets', 'Kafka Queue' y 'Redis Cache'. Un avatar de usuario con gafas de realidad virtual analiza las tuberías con un escáner de rayos X, revelando la lógica matemática oculta en su interior."